CS 344 **Problem Set 1**
Fall 2021 **Due:** October 5, 2021, 11:59 p.m.

---

**Name**: (Xiaowei Zhang) **NetID**: (xz561)

**Honor Code**: Students may discuss and work on homework problems in groups, which is encouraged. However, each student must write down their solutions independently to show they understand the solution well enough to reconstruct it by themselves. Students should clearly mention the names of the other students who offered discussions. We check all submissions for plagiarism. We take the honor code seriously and expect students to do the same.

**Instruction for Submission**: This homework has a total of 100 points, it will be rescaled to 10 points as the eventual score.

We encourage you to use LaTex to write your answer, because it's particularly suitable to type equations and it's frequently used for writing academic papers. We have provided the homework1.tex file for you, you can write your answer to each question in this .tex file directly after the corresponding question, and then compile the .tex file into a PDF file for submission. As a quick introduction to LaTex, please see this *Learn LaTeX in 30 minutes* [1]. Compiling a .tex file into a PDF file needs installing the LaTex software on your laptop. It's free and open source. But if you don't want to install the software, you can just use this website `https://www.overleaf.com/`, which is also free of charge. You can just create an empty project in this website and upload the homework1.zip, and then the website will compile the PDF for you. You can also directly edit your answers on the website and instantly compile your file.

You can also use Microsoft Word or other software if you don't want to use LaTex. If so, please clearly number your answers so that we know which answer corresponds to which question. You also need to save your answers as a PDF file for submission.

Finally, please submit your PDF file only. You should name your PDF file as "Firstname-Lastname-NetID.pdf''.

**Late Policy**: The homework is due on 10/5 (Tuesday) at 11:59pm. We will release the solutions of the homework on Canvas on 10/9 (Saturday) 11:59pm. If your homework is submitted to Canvas before 10/5 11:59pm, there will no late penalty. If you submit to Canvas after 10/5 11:59pm and before 10/9 11:59pm (i.e., before we release the solution), your score will be penalized by $0.9^k$, where $k$ is the number of days of late submission. For example, if you submitted on 10/8, and your original score is 80, then your final score will be $80 * 0.9^3 = 58.32$ for $8 - 5 = 3$ days of late submission. If you submit to Canvas after 10/9 11:59pm (i.e., after we release the solution), then you will earn no score for the homework.

**Make best use of picture in Latex**: If you think some part of your answer is too difficult to type using Latex, you may write that part on paper and scan it as a picture, and then insert that part into Latex as a picture, and finally Latex will compile the picture into the final PDF

[1] `https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes`

output. This will make things easier. Regarding how to insert pictures in Latex, you may refer to the Figure 1 in the following, which is an example of inserting pictures in Latex.

---

Discussion Group (People with whom you discussed ideas used in your answers if any):

I acknowledge and accept the Honor Code. Please type your initials below:
**Signed**: (XZ)

---

1. **Big-O notation.** We have learnt big-O notation to compare the growth rates of functions, this exercise helps you to better understand its definition and properties.

   (a) (12 points) Suppose $n$ is the input size, we have the following commonly seen functions in complexity analysis: $f_1(n) = 1, f_2(n) = \log n, f_3(n) = n, f_4(n) = n \log n, f_5(n) = n^2, f_6(n) = 2^n, f_7(n) = n!$. Intuitively, the growth rate of the functions satisfy $1 < \log n < n < n \log n < n^2 < 2^n < n!$. Prove this is true.

   [**Hint**: You are expected to prove the following asymptotics by using the definition of big-O notation: $1 = O(\log n), \log n = O(n), n = O(n \log n), n \log n = O(n^2), n^2 = O(2^n), 2^n = O(n!)$. **Note**: Chap 3.2 of our textbook provides some math facts in case you need.]

   Answer:

   1.We want to prove that 1 = O(log n). Let c=1 and $n_0$=2. We will show that 1≤log n for all n≥2. Since n≥2, n>0. We can take log of both side of the inequation, we have log n≥log 2. That is, log n≥1

   2. We want to prove that log n = O(n). We claim that for all n≥1, n≥log n. Hence c=$n_0$=1 here. We prove this by induction.

   base case: when n=1, log n=0, n≥log n

   induction: assume n≥log n for n>1, then log(n+1)<log(2n), log(2n)=log 2+log n≤log n+1≤n+1.

   both base case and induction case return n≥log n when c=1=$n_0$, thus Q.E.D

   3.Let c=1, $n_0$=2, we prove that n≤nlog n for all n≥2. since n>0, we can divide both sides of the inequation with n, hence 1≤log n, we already proved 1 = O(log n). Thus, n = O(nlog n)

   4.Same as how we prove log n = O(n), since in that case we have $n_0$=1=c, thus n>0, we can multiply both side of the inequation of log n≤n by n. And we have nlog n ≤$n^2$ for n≥1. Q.E.D

   5.Let c=1 and $n_0$=4. We will show that $n^2 \leq 2^n$ for all n≥2. Take log with base 2 of both sides of the inequation, we get log($n^2$)≤log($2^n$), that us 2log n≤n, that is, log n≤(1/2)n, we can use the technique in subquestion 2 to prove that log n = O(n) as well. Thus, Q.E.D.

   6.Let c=2 and $n_0$=1, we will prove that for all n≥1, $2^n \leq 2(n!)$

   base case: let n=1, 2*1!=2=$2^1$

induction: assume $2^n \leq 2n!$, we want to prove $2^{n+1} \leq 2(n+1)!$, $2^{n+1} = 2 * 2^n \leq 2 * 2 * n! \leq 2 * (n+1) * (n!)$

Thus, $2^n \leq 2(n!)$ for all $n \geq 1$. Q.E.D.

(b) (8 points) Let $f, g : N \to R^+$, prove that $O(f(n) + g(n)) = O(\max\{f(n), g(n)\})$. [**Hint**: The key is $\max\{f(n), g(n)\} \leq f(n) + g(n) \leq 2 \cdot \max\{f(n), g(n)\}$. **Note**: Proving this will help you to understand why we can leave out the insignificant parts in big-O notation and only keep the dominate part, e.g., $O(n^2 + n \log n + n) = O(n^2)$.]

Answer: We need to prove that $f(n)+g(n) = O(max\{f(n), g(n)\})$ and $max\{f(n), g(n)\} = O(f(n) + g(n))$

1.Set c=2, $n_0$=1, we prove that $f(n) + g(n) \leq 2 * max\{f(n), g(n)\}$, this is obvious since $max\{f(n), g(n)\} \geq f(n)$ and $max\{f(n), g(n)\} \geq g(n)$

2.Set c=1, $n_0$=1, we prove that $f(n) + g(n) \geq max\{f(n), g(n)\}$, since $f, g : N- > R^+$, this is always true, as $f(n) + g(n) \geq f(n)$ and $f(n) + g(n) \geq g(n)$

Q.E.D.

2. **Proof of correctness.** (10 points) We have the following algorithm that sorts a list of integers to ascending order. Prove that this algorithm is correct. [**Hint:** You are expected to use mathematical induction to provide a rigorous proof.]

**Algorithm 1:** `Sort a list`

```
Input: Unsorted list A = [a_1, ..., a_n] of n items
Output: Sorted list A' = [a'_1, ..., a'_n] of n items in ascending order
for i = 0, i < n − 1, i++ do
    // Find the minimum element
    min_index = i
    for j = i + 1, j < n, j++ do
        if A[j] < A[min_index] then
            min_index = j
    // Swap the minimum element with the first element
    swap(A, i, min_index)
return A
```

Answer: Proof:We are using induction to prove this algorithm is correct

base: when n=1, this is self-evident

Induction:suppose when given n items, this algorithm outputs the right ascended list.We want to prove that when given n+1 items, it will still output the right sorted list.

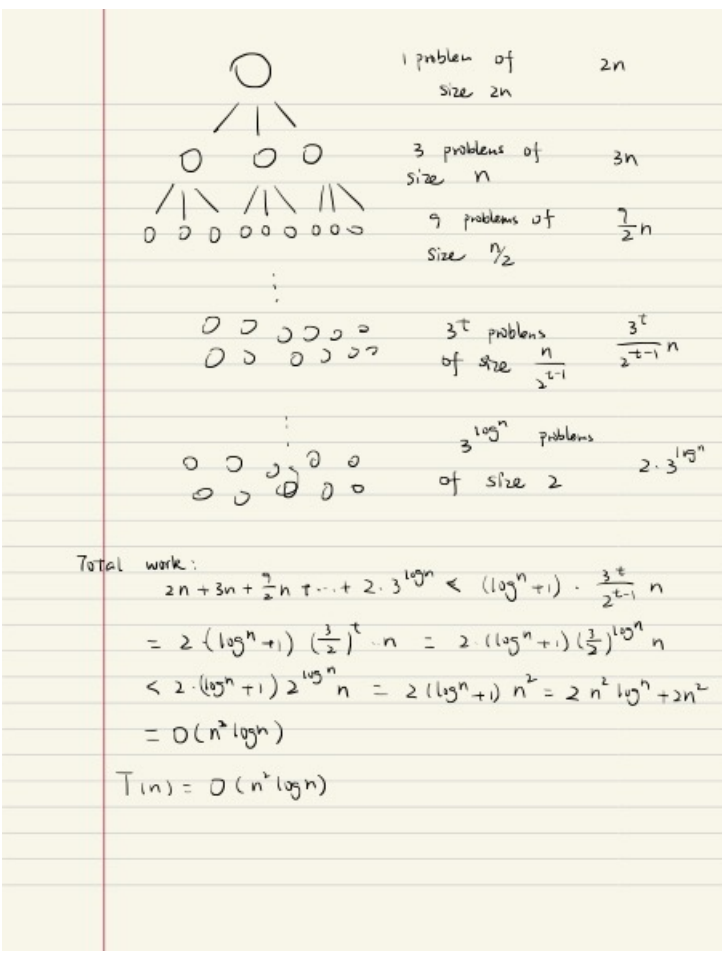
When given n+1 items, i can vary from 0 to n-1, and j would vary from 1 to n. For i≠n-1, and j=n, if A[n]< $A[min_index]$, $min_index$ will be set to n and later A[n] will be swapped with A[i], which is the first item in the rest of the unsorted list.

for i=n-1, it will stay at the end of the list. Thus, given n+1 items, it will output the correct sorted list. Thus, we proved by induction that this algorithm will output the correct answer.

3. **Practice the recursion tree.** (10 points) We have already had a recurrence relation of an algorithm, which is $T(n) = 3T(n/2) + 2n$. Solve this recurrence relation, i.e. express it as $T(n) = O(f(n))$, by using the recursion tree method. [**Note**: If you find it difficult to draw a picture using Latex directly, you can draw it using Microsoft PowerPoint and then save it as a picture file (.png or .jpg), or you can draw on a piece of paper by hand, and take a picture of it using your phone. Then, you can insert the picture into your latex code and compile it into the final PDF submission. The following code shows an example of how to insert figures in latex code, as shown in Figure 1.]

Figure 1: An example showing how to insert figures in latex code.

Answer:



4. **Practice with the iteration method.** We have already had a recurrence relation of an algorithm, which is $T(n) = 4T(n/2) + n^2 \log n$.

   (a) (5 points) Solve this recurrence relation, i.e., express it as $T(n) = O(f(n))$, by using the iteration method.

   Answer:

   (b) (5 points) Prove, by using mathematical induction, that the iteration rule you have observed in 4(a) is correct and you have solved the recurrence relation correctly. [**Hint**: You can write out the general form of $T(n)$ at the iteration step $t$, and prove that this form is correct for any iteration step $t$ by using mathematical induction. Then by finding out the eventual number of $t$ and substituting it into your general form of $T(n)$, you get the $O(\cdot)$ notation of $T(n)$.]

   Answer:

5. **Practice with the Master Theorem**. Solve the following recurrence relations; i.e. express each one as $T(n) = O(f(n))$ for the tightest possible function $f(n)$ using the Master Theorem, and give a short justification. Unless otherwise stated, assume $T(1) = 1$. [**To see the level of detail expected, we have worked out the first one for you.**]

   (z) $T(n) = 6T(n/6) + 1$. We apply the master theorem with $a = b = 6$ and with $d = 0$. We have $a > b^d$, and so the running time is $O(n^{\log_6(6)}) = O(n)$.

   (a) (5 points) $T(n) = 3T(n/4) + \sqrt{n}$

   Answer:

   (b) (5 points) $T(n) = 7T(n/3) + \Theta(n^3)$

   Answer:

   (c) (5 points) $T(n) = 2T(n/3) + n^c$, where $c \geq 1$ is a constant that doesn't depend on $n$.

   Answer:

6. **Proof of the Master Theorem.** (15 points) Now that we have practiced with the recursion tree method, the iteration method, and the Master method. The Master Theorem states that, suppose $T(n) = a \cdot T(n/b) + O(n^d)$, we have:

$$T(n) = \begin{cases} O(n^d \log n), & if\ a = b^d \\ O(n^d), & if\ a < b^d \\ O(n^{\log_b a}), & if\ a > b^d \end{cases}$$

   Prove that the Master Theorem is true by using either the recursion tree method or the iteration method.

   Answer:

7. **Algorithm design.** Each of $n$ users spends some time on a social media site. For each $i = 1, \ldots, n$, user $i$ enters the site at time $a_i$ and leaves at time $b_i \geq a_i$. You are interested in the question: how many distinct pairs of users are ever on the site at the same time? (Here, the pair $(i, j)$ is the same as the pair $(j, i)$).

   Example: Suppose there are 5 users with the following entering and leaving times:

| User | Enter time | Leave time |
|---|---|---|
| 1 | 1 | 4 |
| 2 | 2 | 5 |
| 3 | 7 | 8 |
| 4 | 9 | 10 |
| 5 | 6 | 10 |

Then, the number of distinct pairs of users who are on the site at the same time is three: these pairs are $(1, 2)$, $(4, 5)$, $(3, 5)$. (Drawing the intervals on a number line may make this easier to see).

(a) (10 points) Given input $(a_1, b_1), (a_2, b_2), \ldots, (a_n, b_n)$ as above in no particular order (i.e., not sorted in any way), describe a straightforward algorithm that takes $\Theta(n^2)$-time to compute the number of pairs of users who are ever on the site at the same time, and explain why it takes $\Theta(n^2)$-time. [**We are expecting pseudocode and a brief justification for its runtime.**]

Answer:

(b) (10 points) Give an $\Theta(n \log(n))$-time algorithm to do the same task and analyze its running time. (**Hint:** consider sorting relevant events by time). [**We are expecting pseudocode and a brief justification for its runtime.**]

Answer: